File Handling in Python (Detailed Guide)

File handling in Python is used to:

- Create files
- Read data from files
- Write data into files
- Append data
- Delete files
- Work with different file types like `.txt`, `.csv`, `.json`, etc.

Python provides built-in functions for file operations.

---

1. Opening a File

Python uses the `open()` function.

Syntax

```
file_object = open("filename", "mode")
```

Example

```
f = open("sample.txt", "r")
```

---

2. File Modes

Mode	Description
`"r"`	Read mode (default)
`"w"`	Write mode (overwrites file)
`"a"`	Append mode
`"x"`	Create new file
`"t"`	Text mode
`"b"`	Binary mode
`"r+"`	Read + Write
`"w+"`	Write + Read
`"a+"`	Append + Read

---

3. Reading Files

Suppose `sample.txt` contains:

```
Hello Python
Welcome to File Handling
```

---

a) `read()`

Reads the whole file.

```
f = open("sample.txt", "r")
data = f.read()
print(data)
f.close()
```

Output

```
Hello Python
Welcome to File Handling
```

---

b) `read(n)`

Reads first `n` characters.

```
f = open("sample.txt", "r")
print(f.read(5))
f.close()
```

Output

```
Hello
```

---

c) `readline()`

Reads one line at a time.

```
f = open("sample.txt", "r")
print(f.readline())
print(f.readline())
f.close()
```

---

d) `readlines()`

Returns all lines as a list.

```
f = open("sample.txt", "r")
lines = f.readlines()
print(lines)
```

```
f.close()
```

Output

```
['Hello Python\n', 'Welcome to File Handling']
```

---

4. Writing Files

a) Write Mode (`w`)

Creates file if not exists.
Overwrites existing content.

```
f = open("data.txt", "w")
f.write("Python is awesome")
f.close()
```

---

b) Writing Multiple Lines

```
f = open("data.txt", "w")

lines = ["Apple\n", "Banana\n", "Mango\n"]

f.writelines(lines)

f.close()
```

---

5. Appending Data

Append mode adds data at the end.

```
f = open("data.txt", "a")
f.write("\nNew Line Added")
f.close()
```

---

6. Closing a File

Always close files to free system resources.

```
f.close()
```

---

7. Using `with` Statement (Best Practice)

Automatically closes the file.

Syntax

```
with open("sample.txt", "r") as f:
    data = f.read()
    print(data)
```

Advantages:

- Cleaner code
- Automatic closing
- Prevents memory leaks

---

8. File Pointer Functions

a) `tell()`

Returns current file position.

```
f = open("sample.txt", "r")
print(f.tell())
print(f.read(5))
print(f.tell())
f.close()
```

---

b) `seek()`

Moves cursor to specified position.

```
f = open("sample.txt", "r")
f.seek(6)
print(f.read())
f.close()
```

---

9. Working with Binary Files

Used for images, videos, PDFs, etc.

Writing Binary File

```
f = open("image.jpg", "rb")
data = f.read()
f.close()
```

Copying Binary File

```
with open("image.jpg", "rb") as source:
    with open("copy.jpg", "wb") as target:
        target.write(source.read())
```

---

10. File Existence Checking

Using `os` module.

```
import os

if os.path.exists("sample.txt"):
    print("File exists")
else:
    print("File not found")
```

---

11. Creating and Deleting Files

Create File

```
f = open("newfile.txt", "x")
f.close()
```

Delete File

```
import os

os.remove("newfile.txt")
```

---

12. File Handling Exceptions

Use `try-except`.

```
try:
    f = open("abc.txt", "r")
    print(f.read())

except FileNotFoundError:
    print("File does not exist")

finally:
    print("Done")
```

---

13. Working with CSV Files

Python provides `csv` module.

Writing CSV

```
import csv

with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)

    writer.writerow(["Name", "Age"])
    writer.writerow(["John", 20])
```

Reading CSV

```
import csv

with open("students.csv", "r") as f:
    reader = csv.reader(f)

    for row in reader:
        print(row)
```

---

14. Working with JSON Files

Using `json` module.

Writing JSON

```
import json

data = {
    "name": "Alice",
    "age": 22
}

with open("data.json", "w") as f:
    json.dump(data, f)
```

Reading JSON

```
import json

with open("data.json", "r") as f:
    data = json.load(f)

print(data)
```

---

15. Important File Methods

Method	Description
`read()`	Reads entire file
`readline()`	Reads one line
`readlines()`	Reads all lines
`write()`	Writes string
`writelines()`	Writes list of strings
`seek()`	Moves cursor
`tell()`	Current cursor position
`close()`	Closes file

---

16. Difference Between Modes

Mode	File Exists	File Not Exists
`r`	Read	Error
`w`	Overwrite	Create
`a`	Append	Create
`x`	Error	Create

---

17. Real-World Example

Student Record System

```
while True:
    print("1. Add Student")
    print("2. View Students")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        name = input("Enter student name: ")

        with open("students.txt", "a") as f:
            f.write(name + "\n")

    elif choice == "2":
        with open("students.txt", "r") as f:
            print(f.read())

    elif choice == "3":
        break
```

---

18. Best Practices

✅ Use `with open()`
✅ Handle exceptions
✅ Close files properly
✅ Use correct mode
✅ Use binary mode for non-text files
✅ Validate file existence

---

19. Interview Questions

Q1. Difference between `write()` and `writelines()`?

- `write()` → writes single string
- `writelines()` → writes multiple strings

Q2. Difference between `read()` and `readline()`?

- `read()` → entire file
- `readline()` → one line

Q3. Why use `with open()`?

It automatically closes files.

---

20. Summary

Python file handling allows you to:

- Read files
- Write files
- Append data
- Handle CSV/JSON
- Work with binary files
- Manage file errors efficiently

The most recommended way is:

```
with open("file.txt", "r") as f:
    data = f.read()
```

because it is safe and clean.

Link : CHATGPT

Working With File Paths in Python (Detailed Guide)

A **file path** tells Python where a file or folder is located on your computer.

For example:

```
C:\Users\Krish\Documents\data.txt
```

This is the path of the file `data.txt`.

Why Are File Paths Important?

Python needs the correct path to:

- Read files
- Write files
- Delete files
- Create folders
- Access images, PDFs, CSV files, etc.

Without the correct path, Python cannot find the file.

Types of File Paths

1. Absolute Path

An absolute path specifies the complete location of a file from the root directory.

Windows Example

```
C:\Users\Krish\Documents\data.txt
```

Linux/macOS Example

```
/home/krish/Documents/data.txt
```

Python Example

```
f = open(r"C:\Users\Krish\Documents\data.txt", "r")
print(f.read())
f.close()
```

Advantages

- Exact location
- Works from any current directory

Disadvantages

- Less portable
- Changes from one computer to another

---

2. Relative Path

A relative path specifies the file location relative to the current working directory.

Suppose:

```
Project/
│
├── main.py
└── data.txt
```

Then:

```
f = open("data.txt", "r")
print(f.read())
```

Python looks for `data.txt` in the same folder as the current working directory.

Current Working Directory (CWD)

The current working directory is the folder from which Python is running.

Example:

```
import os

print(os.getcwd())
```

Output:

```
C:\Users\Krish\Project
```

Changing the Current Directory

Use `os.chdir()`.

```
import os

os.chdir("C:\\Users\\Krish\\Documents")

print(os.getcwd())
```

Output:

```
C:\Users\Krish\Documents
```

The os Module

The `os` module provides functions for interacting with the operating system.

```
import os
```

Joining File Paths

Avoid manually writing:

```
path = "folder" + "\\" + "file.txt"
```

Instead use:

```
import os

path = os.path.join("folder", "file.txt")

print(path)
```

Windows Output:

```
folder\file.txt
```

Linux/macOS Output:

```
folder/file.txt
```

Getting File Name from Path

```
import os

path = r"C:\Users\Krish\Documents\data.txt"

print(os.path.basename(path))
```

Output:

```
data.txt
```

Getting Folder Name

```
import os

path = r"C:\Users\Krish\Documents\data.txt"

print(os.path.dirname(path))
```

Output:

```
C:\Users\Krish\Documents
```

Splitting a Path

```
import os

path = r"C:\Users\Krish\Documents\data.txt"

folder, filename = os.path.split(path)

print(folder)
print(filename)
```

Output:

```
C:\Users\Krish\Documents
data.txt
```

Checking Whether a Path Exists

```
import os

path = "data.txt"

print(os.path.exists(path))
```

Output:

```
True
```

or

```
False
```

Checking Whether It Is a File

```
import os

print(os.path.isfile("data.txt"))
```

Output:

```
True
```

Checking Whether It Is a Directory

```
import os

print(os.path.isdir("Documents"))
```

Output:

```
True
```

Getting Absolute Path

```
import os

print(os.path.abspath("data.txt"))
```

Output:

```
C:\Users\Krish\Project\data.txt
```

The pathlib Module (Modern Approach)

Python 3.4 introduced the `pathlib` module.

It provides an object-oriented way to work with paths.

```
from pathlib import Path
```

Creating a Path Object

```
from pathlib import Path

path = Path("data.txt")
```

```
print(path)
```

Output:

```
data.txt
```

Current Working Directory Using pathlib

```
from pathlib import Path

print(Path.cwd())
```

Example Output:

```
C:\Users\Krish\Project
```

Home Directory

```
from pathlib import Path

print(Path.home())
```

Example Output:

```
C:\Users\Krish
```

Joining Paths Using pathlib

```
from pathlib import Path

path = Path("folder") / "subfolder" / "data.txt"

print(path)
```

Windows Output:

```
folder\subfolder\data.txt
```

Linux/macOS Output:

```
folder/subfolder/data.txt
```

Checking Existence

```
from pathlib import Path
```

```
path = Path("data.txt")

print(path.exists())
```

Output:

```
True
```

Checking File or Directory

```
from pathlib import Path

path = Path("data.txt")

print(path.is_file())
print(path.is_dir())
```

Output:

```
True
False
```

Creating Directories

Single Directory

```
from pathlib import Path

Path("Reports").mkdir()
```

Creates:

```
Reports/
```

Nested Directories

```
from pathlib import Path

Path("Data/2026/June").mkdir(parents=True, exist_ok=True)
```

Creates:

```
Data/
└── 2026/
    └── June/
```

Reading and Writing Files Using pathlib

Writing

```
from pathlib import Path

path = Path("notes.txt")

path.write_text("Hello Python")
```

Reading

```
from pathlib import Path

text = path.read_text()

print(text)
```

Output:

```
Hello Python
```

File Extension

```
from pathlib import Path

path = Path("report.pdf")

print(path.suffix)
```

Output:

```
.pdf
```

File Name Without Extension

```
from pathlib import Path

path = Path("report.pdf")

print(path.stem)
```

Output:

```
report
```

Changing File Extension

```
from pathlib import Path

path = Path("report.txt")

new_path = path.with_suffix(".pdf")

print(new_path)
```

Output:

```
report.pdf
```

Listing Files in a Folder

```
from pathlib import Path

for file in Path(".").iterdir():
    print(file)
```

Output Example:

```
main.py
data.txt
notes.txt
```

Finding Specific Files

Find all `.txt` files:

```
from pathlib import Path

for file in Path(".").glob("*.txt"):
    print(file)
```

Recursive Search

Find `.py` files inside all subfolders:

```
from pathlib import Path

for file in Path(".").rglob("*.py"):
    print(file)
```

---

os vs pathlib

Feature	os.path	pathlib
Introduced	Earlier versions	Python 3.4
Style	Function-based	Object-oriented
Readability	Moderate	Excellent
Join Paths	`os.path.join()`	`/` operator
Recommended	Good	Best for new projects

Real-World Example

Suppose your project structure is:

```
Project/
├── main.py
├── data/
│   └── students.csv
└── reports/
```

Read the CSV file:

```
from pathlib import Path

csv_path = Path("data") / "students.csv"

if csv_path.exists():
    print(csv_path.read_text())
else:
    print("File not found")
```

Best Practices

✅ Use relative paths inside projects.

✅ Prefer `pathlib` for new projects.

✅ Use `Path.exists()` before accessing files.

✅ Avoid hardcoding Windows paths.

✅ Use `os.path.join()` or `Path()` instead of string concatenation.

Interview Questions

1. What is the difference between absolute and relative paths?

- **Absolute Path:** Complete path from the root directory.
- **Relative Path:** Path relative to the current working directory.

---

2. Why is `pathlib` preferred?

Because it is more readable, cross-platform, and object-oriented.

---

3. What does `os.getcwd()` do?

It returns the current working directory.

Summary

Working with file paths helps Python locate and manage files correctly. The two most common approaches are:

- **`os` module** → Traditional method.
- **`pathlib` module** → Modern and recommended method.

For modern Python development, prefer:

```
from pathlib import Path

path = Path("data") / "file.txt"
```

because it is cleaner, easier to read, and works across different operating systems.

Link : CHATGPT

Link : ALL